

Java Practice — Approach & Solutions

Companion to Java_Questions.pdf

Section A — Theory Questions

Q1. What is Java? List any three key features of Java.

Approach: Recall Java's definition and pick distinguishing features that set it apart from other languages.

Answer:

Java is a high-level, object-oriented, platform-independent programming language. Three key features: (1) Platform Independence — compiled code (bytecode) runs on any system with a JVM ('Write Once, Run Anywhere'). (2) Object-Oriented — everything is organized around classes and objects. (3) Robust & Secure — strong memory management, automatic garbage collection, and no explicit pointers.

Q2. Differentiate between JDK, JRE, and JVM.

Approach: Think of them as layered components: JVM runs code, JRE provides the environment to run, JDK provides tools to develop.

Answer:

JVM (Java Virtual Machine) is the engine that executes Java bytecode. JRE (Java Runtime Environment) includes the JVM plus libraries needed to run Java applications. JDK (Java Development Kit) includes the JRE plus development tools such as the compiler (javac) needed to write and compile Java programs.

Q3. Difference between primitive and non-primitive data types.

Approach: Compare how each stores data and recall standard examples.

Answer:

Primitive data types store actual values directly in memory and have a fixed size (e.g., int, double, char, boolean). Non-primitive (reference) data types store the memory address of the actual data and their size depends on the object (e.g., String, Array, custom classes like Student).

Q4. Difference between = and == in Java.

Approach: Recall operator category: assignment vs comparison.

Answer:

The = operator is the assignment operator; it assigns the value on the right to the variable on the left (e.g., int x = 5;). The == operator is a relational/equality operator; it compares two values and returns a boolean (true/false) indicating whether they are equal (e.g., x == 5).

Q5. Implicit vs explicit type casting.

Approach: Identify direction of conversion: smaller-to-larger happens automatically; larger-to-smaller needs a manual cast.

Answer:

```
// Implicit (widening) - done automatically by Java
int num = 10;
double d = num;    // int -> double, no data loss

// Explicit (narrowing) - must be done manually
double price = 99.99;
int rounded = (int) price;    // double -> int, decimal part lost
```

Q6. while vs do-while loop.

Approach: Compare when the condition is checked relative to the loop body execution.

Answer:

In a **while** loop, the condition is checked first; if false initially, the body never executes. In a **do-while** loop, the body executes first and the condition is checked afterward, so the body always executes at least once. A do-while is preferred when the loop body must run at least one time regardless of the condition, e.g., displaying a menu before asking the user whether to continue.

Q7. break vs continue.

Approach: Think about what each statement does to the loop's control flow.

Answer:

break immediately terminates the loop entirely and control moves to the statement after the loop.
continue skips only the current iteration and moves control to the next iteration of the loop (re-checking the loop condition).

Q8. Why are arrays a reference type?

Approach: Recall how array variables relate to the actual data in memory.

Answer:

An array in Java is a collection of elements of the same type, stored in contiguous memory. It is a reference type because the array variable itself does not hold the elements directly — it holds a reference (memory address) pointing to where the array object (its elements) is actually stored on the heap.

Q9. Default values in an array.

Approach: Recall Java's default initialization rules for declared-but-unassigned variables, applied to array elements.

Answer:

(a) For an int array, every uninitialized element defaults to **0**. (b) For a String array (a reference type), every uninitialized element defaults to **null**, since no object has been assigned yet.

Q10. 1-D array vs 2-D array.

Approach: Compare structure (single row of data vs grid) and map to real-world analogies.

Answer:

A 1-D array is a single list of elements accessed by one index, e.g., `int[] marks` — useful for storing marks of one subject for several students. A 2-D array is an array of arrays (a grid/table) accessed by row and column indices, e.g., `int[][] seatingChart` — useful for representing a classroom seating layout or a tic-tac-toe board.

Q11. What is a method? Components of a method signature.

Approach: Recall the building blocks that make up how a method is identified and called.

Answer:

A method is a named block of code that performs a specific task and can be called/reused. The method signature consists of the method name and the parameter list (number, order, and types of parameters). Access modifier and return type are part of the method declaration but technically not part of the signature used for overloading resolution.

Q12. void method vs method with return type.

Approach: Identify whether the method sends a value back to the caller.

Answer:

```
// void method - performs an action, returns nothing
public void displayMessage() {
    System.out.println("Hello!");
}

// method with return type - sends a value back
public int getSquare(int n) {
    return n * n;
}
```

Q13. Actual parameters vs formal parameters.

Approach: Distinguish between values passed at the call site and variables declared in the method definition.

Answer:

Formal parameters are the variables listed in the method's declaration that receive incoming values, e.g., in `'void add(int a, int b)'`, `a` and `b` are formal parameters. Actual parameters (arguments) are the real values or variables passed when the method is called, e.g., in `'add(5, 10)'`, `5` and `10` are actual parameters.

Q14. Purpose of return; multiple return statements.

Approach: Think about control flow exit points within conditional branches.

Answer:

The **return** keyword sends a value back to the caller and immediately ends the method's execution at that point. A method CAN have multiple return statements as long as they lie in different conditional branches (e.g., inside `if/else` blocks), because only one of them will actually execute for any given call; however, every possible path must lead to exactly one return being reached if the method has a non-void return type.

Q15. Call by value vs call by reference in Java.

Approach: Consider what is actually copied when passing primitives versus passing objects/arrays.

Answer:

In call by value, a copy of the actual value is passed, so changes inside the method do not affect the original variable — Java uses this for primitive types (int, double, etc.). In call by reference, the memory address is passed, so changes inside the method affect the original data — Java effectively achieves this for arrays and objects because the reference (address) itself is passed by value, but it still points to the same underlying array/object, so modifying its contents affects the original.

Section B — Arrays (Declaration & Initialization as Assignment Statements)

Q16. Declare an integer array 'marks' of size 5 (no initial values).

Approach: Use 'new' with the type and size in square brackets to create the array; this is an assignment of a new array object to the variable.

Answer:

```
int[] marks = new int[5];
```

Q17. Declare and initialize a String array 'days' with Mon, Tue, Wed.

Approach: Use array literal syntax with curly braces directly as the assigned value.

Answer:

```
String[] days = {"Mon", "Tue", "Wed"};
```

Q18. Declare a 2-D integer array 'matrix' with 3 rows and 4 columns.

Approach: Specify both dimensions inside separate square brackets when creating the array.

Answer:

```
int[][] matrix = new int[3][4];
```

Q19. Set first and last elements of arr (size 5) to 10 and 50.

Approach: Index 0 is the first element; size-1 (i.e., 4) is the last element. Assign values using index notation.

Answer:

```
arr[0] = 10;  
arr[4] = 50;
```

Q20. Create a double array 'prices' directly from literal values.

Approach: Assign array literal values of type double directly without using 'new' explicitly.

Answer:

```
double[] prices = {99.5, 45.0, 120.75};
```

Q21. Assign the length of array 'data' to integer variable 'len'.

Approach: Arrays expose a public field called 'length' (not a method) that stores the number of elements.

Answer:

```
int len = data.length;
```

Q22. Copy the value of arr[2] into a new variable 'third'.

Approach: Access the element using its index and assign it to a new variable of a matching type.

Answer:

```
int third = arr[2];
```

Section C — Method Declaration

Q23. General syntax for declaring a method in Java.

Approach: Break the declaration line into its labelled components in order.

Answer:

```
accessModifier returnType methodName ( parameterList ) {  
    // method body  
}
```

// Example:

```
public      int      add      (int a, int b) {  
    return a + b;  
}  
// public = access modifier, int = return type,  
// add = method name, (int a, int b) = parameter list
```

Q24. Declare a method 'square' that accepts an int and returns its square.

Approach: Choose return type int since the square of an int is an int; name the parameter and specify its type.

Answer:

```
public int square(int n) {  
    return n * n;  
}
```

Q25. Declare a void method 'greet' that takes a String 'name' and prints a greeting.

Approach: Since nothing is returned, use void as the return type; take one String parameter.

Answer:

```
public void greet(String name) {  
    System.out.println("Hello, " + name + "!");  
}
```

Q26. Declare a method 'isEven' that accepts an int and returns a boolean.

Approach: Use the modulus operator to check divisibility by 2 and return the boolean result.

Answer:

```
public boolean isEven(int n) {  
    return n % 2 == 0;  
}
```

Q27. Declare a method 'sumArray' that accepts an int array and returns the total sum.

Approach: Parameter type is int[]; loop through elements internally and return an int total.

Answer:

```
public int sumArray(int[] arr) {  
    int sum = 0;  
    for (int i = 0; i < arr.length; i++) {  
        sum += arr[i];  
    }  
    return sum;  
}
```

Q28. What happens if a non-void method has no return statement on some path?

Approach: Think about how the compiler verifies that a value is always returned.

Answer:

Java performs compile-time checking called 'reachability analysis' for return statements. If a method is declared with a non-void return type and there exists any possible execution path that does not end in a return statement, the code will **not compile** — the compiler raises the error: "missing return statement".

Section D — Coding / Programming Questions

Q29. Swap two numbers using a third variable.

Approach: Store one value temporarily in a third variable so it isn't lost when overwritten.

Answer:

```
public class SwapNumbers {  
    public static void main(String[] args) {  
        int a = 5, b = 10, temp;  
        temp = a;  
        a = b;  
        b = temp;  
        System.out.println("a = " + a + ", b = " + b);  
    }  
}
```

Q30. Check whether a number is prime.

Approach: A number is prime if it has no divisors other than 1 and itself. Loop from 2 to $n/2$ (or $\sqrt{n}$) checking divisibility.

Answer:

```
public class PrimeCheck {
    public static void main(String[] args) {
        int n = 29;
        boolean isPrime = true;
        if (n < 2) {
            isPrime = false;
        } else {
            for (int i = 2; i <= n / 2; i++) {
                if (n % i == 0) {
                    isPrime = false;
                    break;
                }
            }
        }
        System.out.println(n + " is prime: " + isPrime);
    }
}
```

Q31. Find the largest and smallest element in an array of 6 elements.

Approach: Initialize both max and min to the first element, then traverse the array comparing and updating each.

Answer:

```
public class MinMaxArray {
    public static void main(String[] args) {
        int[] nums = {12, 45, 3, 67, 23, 9};
        int max = nums[0], min = nums[0];
        for (int i = 1; i < nums.length; i++) {
            if (nums[i] > max) max = nums[i];
            if (nums[i] < min) min = nums[i];
        }
        System.out.println("Largest: " + max);
        System.out.println("Smallest: " + min);
    }
}
```

Q32. Calculate sum and average of array elements.

Approach: Add all elements in a loop to get the sum, then divide by array length (as a double) for the average.

Answer:

```
public class SumAverage {
    public static void main(String[] args) {
        int[] nums = {10, 20, 30, 40, 50};
        int sum = 0;
        for (int i = 0; i < nums.length; i++) {
            sum += nums[i];
        }
        double average = (double) sum / nums.length;
        System.out.println("Sum: " + sum);
        System.out.println("Average: " + average);
    }
}
```

Q33. Reverse the elements of an array without a built-in method.

Approach: Swap elements from both ends moving toward the middle using a two-pointer approach.

Answer:

```
public class ReverseArray {
    public static void main(String[] args) {
        int[] arr = {1, 2, 3, 4, 5};
        int start = 0, end = arr.length - 1;
        while (start < end) {
            int temp = arr[start];
            arr[start] = arr[end];
            arr[end] = temp;
            start++;
            end--;
        }
        for (int num : arr) {
            System.out.print(num + " ");
        }
    }
}
```

Q34. Sum of diagonal elements of a 3x3 matrix using a 2-D array.

Approach: Diagonal elements occur where row index equals column index (matrix[i][i]). Loop once through rows and add that element.

Answer:

```
public class DiagonalSum {
    public static void main(String[] args) {
        int[][] matrix = {
            {1, 2, 3},
            {4, 5, 6},
            {7, 8, 9}
        };
        int sum = 0;
        for (int i = 0; i < matrix.length; i++) {
            sum += matrix[i][i];
        }
        System.out.println("Diagonal Sum: " + sum);
    }
}
```

Q35. Method factorial(int n) returning factorial of n, called for n = 5.

Approach: Use an iterative loop multiplying numbers from 1 to n, accumulating the result, then return it.

Answer:

```
public class FactorialDemo {
    static int factorial(int n) {
        int result = 1;
        for (int i = 1; i <= n; i++) {
            result *= i;
        }
        return result;
    }

    public static void main(String[] args) {
        int n = 5;
        System.out.println("Factorial of " + n + " = " + factorial(n));
    }
}
```

Q36. Method `maxOfThree(int a, int b, int c)` returning the largest of three numbers.

Approach: Compare the three values using nested or chained conditional logic and return the greatest.

Answer:

```
public class MaxOfThree {
    static int maxOfThree(int a, int b, int c) {
        int max = a;
        if (b > max) max = b;
        if (c > max) max = c;
        return max;
    }

    public static void main(String[] args) {
        System.out.println("Max: " + maxOfThree(15, 42, 8));
    }
}
```

Q37. Void method `printTable(int n)` printing the multiplication table of n (1 to 10).

Approach: Use a for loop from 1 to 10, printing n multiplied by the loop counter each iteration; no return value needed since the task is just to print.

Answer:

```
public class MultiplicationTable {
    static void printTable(int n) {
        for (int i = 1; i <= 10; i++) {
            System.out.println(n + " x " + i + " = " + (n * i));
        }
    }

    public static void main(String[] args) {
        printTable(7);
    }
}
```

Q38. Method `countPositives(int[] arr)` returning count of positive numbers; declare an array of 5 integers and pass it to the method.

Approach: Declare/initialize the array first (the assignment step), then pass the whole array as an argument; inside the method, loop and increment a counter for elements greater than 0.

Answer:

```
public class CountPositives {
    static int countPositives(int[] arr) {
        int count = 0;
        for (int i = 0; i < arr.length; i++) {
            if (arr[i] > 0) {
                count++;
            }
        }
        return count;
    }

    public static void main(String[] args) {
        int[] numbers = {-3, 7, 12, -8, 5};
        int result = countPositives(numbers);
        System.out.println("Positive numbers count: " + result);
    }
}
```

End of Solutions.